

Módulo de Solicitudes — SIIAA Laravel 13

1. Resumen ejecutivo

El módulo de **Solicitudes** del SIIAA Laravel 13 será una actualización y rediseño funcional del módulo existente en el **SIIAA_10**. Su objetivo principal es permitir el registro, edición, consulta, envío, revisión operativa, seguimiento administrativo, archivo lógico y eventual migración histórica de solicitudes institucionales relacionadas con permisos de ausencia, recursos, visitantes y apoyos para estudiantes.

El rediseño se basa en mantener la lógica operativa que ya funciona en el sistema anterior, pero adaptándola a la arquitectura nueva del SIIAA, particularmente a:

- Laravel 13.
- Livewire.
- Componentes Blade UI propios del SIIAA.
- Catálogos institucionales centralizados mediante `catalogos_items`.
- Capa de identidad institucional basada en `IdentityLink`.
- Reglas de permisos simplificadas.
- Mejor separación de responsabilidades.
- Mejor rendimiento en listados y consultas.
- Flujo más claro para usuarios propietarios, revisores y administradores.

Este módulo será de uso diario, por lo que el diseño prioriza **usabilidad, mantenimiento, velocidad de captura y consistencia institucional**. Se evita sobrecargar al usuario con formularios excesivos y se favorecen campos libres cuando el catálogo pueda convertirse en una barrera operativa.

2. Objetivo del módulo

El módulo permitirá registrar y gestionar solicitudes institucionales de los siguientes tipos:

1. **Permiso de ausencia con recursos**
2. **Permiso de ausencia sin recursos**
3. **Solo recursos**
4. **Visitante**
5. **Solicitud de recursos al IRyA para estudiantes SIIAP**

El módulo deberá permitir que:

- Un usuario propietario registre y gestione sus propias solicitudes.
 - Un administrador o rol autorizado cree solicitudes a nombre de otra identidad institucional.
 - Un perfil operativo revise solicitudes y edite campos específicos, como observaciones SACAD o administración.
 - Se registren recursos, documentos, visitantes y requerimientos asociados.
 - Se conserve histórico mediante archivo lógico.
 - Posteriormente se migren datos históricos desde el SIIAA_10.
-

3. Principios de diseño

3.1 Usabilidad

El módulo debe ser rápido y sencillo para usuarios que lo utilizarán con frecuencia. Por ello:

- Se evita pedir campos innecesarios.
- Los documentos se suben de forma natural mediante drag & drop o selector tradicional.
- No se obliga a clasificar documentos.
- Las instituciones se capturan como texto libre en solicitudes generales.
- Los catálogos se usan donde sí aportan control institucional.
- Las validaciones dependen del tipo de solicitud y motivo, no de un formulario rígido universal.

3.2 Mantenibilidad

El diseño evita atomicidad excesiva cuando no es necesaria. Por ejemplo:

- No se crea una tabla independiente de observaciones.
- Las observaciones SACAD y administración viven directamente en `solicitudes`.
- Visitantes sí se separan en tabla hija porque tienen lógica propia.
- Requerimientos de visitante cuelgan de visitantes, no de solicitudes generales.
- Recursos conservan una estructura cercana al sistema anterior porque ya funciona operativamente.

3.3 Consistencia institucional

Todas las referencias de propiedad, autoría y auditoría deben resolverse mediante `IdentityLink`, no directamente mediante `User`.

Esto aplica a:

- `owner_id`
- `created_by`
- `updated_by`
- `archived_by`
- `politica_aceptada_by`
- `uploaded_by`
- `tutor_id`
- campos equivalentes de auditoría

4. Identidad institucional

El módulo debe operar sobre la capa de identidad del SIIAA.

4.1 Propietario de la solicitud

Cada solicitud tendrá un propietario institucional:

```
solicitudes.owner_id → identity_links.id
```

El propietario puede ser:

- Persona IRyA.
- Estudiante SIIAP.
- Otra identidad institucional futura, si el SIIAA la contempla.

4.2 Creación propia y creación a nombre de otra persona

Un usuario puede tener doble condición, por ejemplo:

- Investigador y administrador.
- Académico y gestor.
- Usuario con identidad propia y permiso de gestión.

Por ello, la creación de solicitudes se divide en dos posibilidades:

Caso	Regla
Crear para mí	Disponible si el usuario tiene una <code>IdentityLink</code> válida
Crear a nombre de otra persona	Disponible si el usuario tiene <code>solicitudes.manage</code>

Cuando un gestor crea una solicitud a nombre de otra persona:

```
owner_id = identidad representada  
created_by = identidad activa del gestor
```

Si el gestor crea una solicitud para sí mismo:

```
owner_id = identidad propia  
created_by = identidad propia
```

5. Permisos

El módulo usará una estructura de permisos reducida.

5.1 Permisos principales

Permiso	Alcance
<code>solicitudes.access</code>	Permite al usuario crear, ver, editar y borrar solo sus propias solicitudes, según estado

Permiso	Alcance
<code>solicitudes.review</code>	Permite ver todas las solicitudes y editar campos específicos de revisión/operación
<code>solicitudes.manage</code>	Permite gestión completa: ver todo, crear a nombre de otra identidad, editar, cambiar estados, borrar y archivar

5.2 Permiso para estudiantes asociados

Para el mini módulo de estudiantes asociados se usará solo:

```
estudiantes-asociados.manage
```

Este permiso permite administración completa del mini módulo.

Los usuarios generales no tienen edición libre del mini módulo, pero sí podrán buscar, seleccionar o crear estudiante asociado desde el flujo de una solicitud de visitante cuando aplique.

6. Estados de solicitud

6.1 Estados finales

Los estados finales del flujo son:

Clave	Estado
<code>BORRADOR</code>	Borrador
<code>SENV</code>	Enviada
<code>APRCI</code>	Aprobada en CI
<code>RECI</code>	Rechazada en CI
<code>TRPAG</code>	En trámite de pago
<code>PAG</code>	Pagada
<code>CLO</code>	Cerrada
<code>CANCELADA</code>	Cancelada

También existe el estado histórico:

Clave	Estado
<code>REVCIC</code>	En revisión CIC

`REVCIC` se conserva por compatibilidad histórica, pero no forma parte del flujo normal del nuevo módulo.

6.2 Estados descartados del flujo nuevo

No se usarán como estados funcionales nuevos:

EN_REVISION
OBSERVADA

Las correcciones solicitadas por SACAD o administración se manejarán mediante:

observaciones_sacad
observaciones_administracion

sin cambiar a un estado intermedio.

6.3 Regla de edición del propietario

Estado	Editable por propietario
BORRADOR	Sí
SENV / Enviada	Sí
APRCI	No
RECI	No
TRPAG	No
PAG	No
CLO	No
CANCELADA	No

El propietario solo puede transicionar:

BORRADOR → ENVIADA

Los demás cambios de estado quedan para usuarios con `solicitudes.manage`.

7. Tipos de solicitud

Los tipos finales de solicitud son:

Clave catálogo	ID	Uso funcional
AUS_REC	554	Permiso de ausencia con recursos

Clave catálogo	ID	Uso funcional
AUSENCIA	517	Permiso de ausencia sin recursos
SOLOREC	518	Solo recursos
VISITA	325	Visitante
ESOLREC	516	Solicitud de recursos al IRyA para estudiantes

No existe tipo de solicitud **OTRO**.

La opción **OTRO** aplica únicamente como **motivo de solicitud**.

8. Motivos de solicitud

Los motivos provienen del catálogo **SOLMOT**.

Clave	ID	Motivo
EVACAD	323	Evento académico
ESTT	324	Estancia de Trabajo
ACTDIV	328	Actividad de divulgación
TCAMP	329	Trabajo de campo
OTRO	555	Otro / especificar

8.1 Regla de aplicación

Todos los motivos aplican a todos los tipos de solicitud excepto:

VISITA

Las solicitudes de visitante tienen su propia lógica mediante **solicitudes_visitantes**.

8.2 Motivo **OTRO**

Cuando el motivo sea **OTRO**:

motivo_otro

será obligatorio.

motivo_otro funcionará como la descripción principal de la actividad, necesidad o motivo. En la interfaz se deberá recomendar al usuario ser muy específico.

```
informacion_adicional
```

queda como campo opcional para contexto extra.

9. Flujo general del módulo

El módulo conserva un flujo de cuatro pasos:

```
Paso 1. Datos generales  
Paso 2. Recursos  
Paso 3. Documentos  
Paso 4. Revisión y envío
```

9.1 Paso 1 — Datos generales

En este paso se capturan:

- Propietario.
- Tipo de solicitud.
- Motivo, excepto visitante.
- Motivo otro, si aplica.
- Fechas.
- País.
- Nombre del evento.
- Tipo de presentación.
- Institución.
- Anfitrión.
- Lugar.
- Tutor, si aplica.
- Información adicional.
- Seguro UNAM, si aplica.
- Datos de visitante, si el tipo de solicitud es visitante.

9.2 Paso 2 — Recursos

Este paso aparece solo cuando:

```
requiere_recursos = true
```

Aplica a:

Tipo	Requiere recursos
AUS_REC	Sí
AUSENCIA	No

Tipo	Requiere recursos
SOLOREC	Sí
ESOLREC	Sí
VISITA	Seleccionable

9.3 Paso 3 — Documentos

Los documentos son flexibles:

- El paso siempre existe.
- No bloquea el envío si no hay documentos.
- Se muestra advertencia si no se adjuntan archivos.
- SACAD o administración pueden solicitar documentación posteriormente mediante observaciones.

9.4 Paso 4 — Revisión y envío

Este paso muestra un resumen de la solicitud y valida:

- Propietario válido.
- Tipo válido.
- Motivo válido, excepto visitante.
- Motivo otro si aplica.
- Datos requeridos por tipo y motivo.
- Recursos si `requiere_recursos = true`.
- Política de recursos aceptada si aplica.
- Visitante completo si aplica.
- Documentos solo como advertencia, no como bloqueo.

Al enviar se asigna folio y estado `SENV`.

10. Folio institucional

10.1 Regla final

El folio institucional se asigna hasta el envío formal de la solicitud.

Mientras la solicitud esté en borrador:

```
folio = null
folio_year = null
folio_number = null
```

En interfaz se mostrará algo como:

Folio pendiente

o:

Se asignará al enviar

10.2 Formato

YYYY/NNN

Ejemplo:

2026/001
2026/002
2026/003

10.3 Consecutivo anual

Se usarán campos separados:

folio_year
folio_number
folio

con restricción:

`unique(folio_year, folio_number)`

10.4 No reutilización

Los folios de solicitudes ya enviadas no se reasignan ni reutilizan aunque la solicitud sea:

- Cancelada.
- Cerrada.
- Archivada.
- Borrada administrativamente.

Estos folios se consideran consumidos para preservar trazabilidad institucional.

11. Tabla principal solicitudes

11.1 Estructura conceptual

```
solicitudes
- id

- folio
- folio_year
- folio_number

- owner_id
- created_by
- updated_by

- tipo_solicitud_id
- motivo_id
- motivo_otro
- requiere_recursos

- fecha_inicio
- fecha_fin
- pais_id
- nombre_evento
- tipo_presentacion
- institucion
- anfitrión
- lugar
- tutor_id
- informacion_adicional

- requiere_seguro_unam
- seguro_unam_beneficiario

- observaciones_sacad
- observaciones_administracion

- estatus_id
- submitted_at
- approved_at
- rejected_at
- closed_at
- cancelled_at

- politica_aceptada_at
- politica_aceptada_by
- politica_version

- archived_at
- archived_by
```

- archive_reason
- created_at
- updated_at

11.2 Institución

Para solicitudes generales no visitantes se usará solo:

institucion

como campo libre.

No se usará `institucion_id` en la tabla `solicitudes`.

Esto aplica a:

- Ausencia con recursos.
- Ausencia sin recursos.
- Solo recursos.
- Recursos IRyA.

11.3 Tutor

tutor_id

apunta conceptualmente a `identity_links.id`.

En interfaz se selecciona mediante buscador filtrando identidades elegibles:

- Investigadores.
- Técnicos académicos / académicos.
- Posdocs.

11.4 Seguro UNAM

Campos:

requiere_seguro_unam
seguro_unam_beneficiario

Si se activa el checkbox de seguro, se muestra y solicita el campo de beneficiario/datos del seguro.

Aplica principalmente a:

- Actividades que impliquen salir del instituto.
- Trabajo de campo.

- Permisos de ausencia.
- Visitantes cuando corresponda.

12. Recursos

12.1 Tabla solicitudes_recursos

```
solicitudes_recursos
- id
- solicitud_id

- origen_id
- proyecto_id
- proyecto_nombre

- dias_n
- dias_i

- cuota
- cuota_divisa
- avion
- avion_divisa
- otro
- otro_divisa

- informacion_adicional

- created_by
- updated_by
- created_at
- updated_at
```

12.2 Campos monetarios

Los campos monetarios usarán:

```
decimal(12,2)
```

Aplica a:

- cuota
- avion
- otro

12.3 Política de recursos

La política de recursos vive en `solicitudes`, no en cada recurso.

Campos:

```
politica_aceptada_at  
politica_aceptada_by  
politica_version
```

Regla:

```
Si requiere_recursos = true, antes de enviar debe existir:  
- al menos un recurso  
- política aceptada
```

12.4 Orígenes de recurso

Catálogo **C_OREC**:

Clave	ID	Origen
R_PI	286	Partida individual
R_PAPIIT	287	Proyecto PAPIIT
R_PAPIME	288	PAPIME
CONV	327	Convenio
R_IRYA	522	Recursos del IRyA
SECIHT	539	SECIHTI

12.5 Divisas

Catálogo **DIVISAS**:

Clave	ID	Divisa
MXN	519	Pesos Mexicanos
USD	520	Dólares EEUU
EUR	521	Euros

MXN debe ser default.

13. Documentos

13.1 Tabla solicitudes_documentos

```
solicitudes_documentos
- id
- solicitud_id

- filename
- original_name
- path
- mime_type
- size

- uploaded_by
- created_at
- updated_at
```

13.2 Interfaz

La carga de documentos debe ser ligera:

- Drag & drop.
- Selector tradicional.
- Sin campos adicionales por archivo.
- Validación automática de formato y tamaño.
- Recomendación al usuario para usar nombres descriptivos.

13.3 Ruta de almacenamiento

Ruta final aprobada:

```
documentos/solicitudes/{year}/{solicitud_id}/
```

Ejemplo:

```
documentos/solicitudes/2026/125/
```

Ventajas:

- Funciona desde borrador.
- No depende del folio.
- No requiere mover archivos al enviar.
- El año ayuda para auditoría.
- El folio se consulta desde base de datos.

14. Visitantes

14.1 Tabla solicitudes_visitantes

```
solicitudes_visitantes
- id
- solicitud_id

- tipo_visitante_id
- estudiante_asociado_id

- nombre
- apellidos
- email
- pais_id
- institucion_id
- institucion
- lugar
- fecha_inicio
- fecha_fin

- created_at
- updated_at
```

14.2 Regla general

Una solicitud de visitante tendrá un solo visitante principal.

Si se requiere más de un visitante, se deberán crear solicitudes separadas.

14.3 Institución en visitantes

Para visitantes sí se usará estructura flexible:

```
institucion_id nullable
institucion nullable
```

Esto permite seleccionar una institución de catálogo si existe, pero también capturar texto libre si no existe.

14.4 Tipos de visitante

Catálogo C_SOLTVIS:

Clave	ID	Tipo
VACAD	523	Académico / Investigador

Clave	ID	Tipo
VEASOC	524	Estudiante asociado
VEST	525	Estudiante no asociado
VOTRO	526	Otro

14.5 Visitante estudiante asociado

Si el visitante es estudiante asociado:

estudiante_asociado_id

es obligatorio.

El registro formal vive en:

estudiantes_asociados
estudiantes_asociados_ingresos

La solicitud solo guarda el vínculo.

15. Requerimientos de visitante

15.1 Tabla solicitudes_requerimientos

```
solicitudes_requerimientos
- id
- solicitud_visitante_id
- requerimiento_id

- created_by
- updated_by
- created_at
- updated_at
```

15.2 Reglas

- Cuelgan de solicitudes_visitantes, no de solicitudes.
- Son opcionales.
- Se capturan como checkboxes.
- No hay opción Otro.
- No hay campo de información adicional.
- Una visita puede no tener requerimientos.

15.3 Catálogo VIS_REQ

Clave	ID	Requerimiento
REQ_OF	279	Requiere oficina
REQ_CCOP	280	Requiere cuenta de cómputo
REQ_ECOP	281	Requiere equipo de cómputo
REQ_ACER	282	Requiere acceso al acervo
D_COLOQ	283	Tiene disponibilidad para impartir coloquio

16. Estudiantes asociados

16.1 Naturaleza del submódulo

Los estudiantes asociados trabajan ligados a solicitudes de visitante, pero deben existir como mini módulo administrativo independiente.

Tablas relacionadas:

```
estudiantes_asociados
estudiantes_asociados_ingresos
```

16.2 Reglas de operación

Acción	Usuario general	Admin
Buscar estudiante asociado desde solicitud	Sí	Sí
Crear estudiante asociado desde solicitud	Sí	Sí
Registrar ingreso desde solicitud	Sí	Sí
Editar ingreso registrado por él	Sí, solo si el periodo no concluyó	
Editar datos generales desde mini módulo	No	Sí
Agregar ingresos libremente desde mini módulo	No	Sí
Editar ingresos vencidos	No	Sí
Activar/desactivar estudiante asociado	No	Sí

16.3 Permiso

Solo se agrega un permiso específico:

```
estudiantes-asociados.manage
```

16.4 Recursos y estudiantes asociados

Un estudiante asociado no puede solicitar recursos directamente.

La solicitud `ESOLREC` es exclusiva para estudiantes SIIAP adscritos al IRyA.

Si un estudiante asociado requiere recursos, la solicitud debe hacerla su tutor o responsable institucional.

17. Observaciones

No se creará tabla `solicitudes_observaciones` en esta versión.

Las observaciones estarán directamente en `solicitudes`:

```
observaciones_sacad  
observaciones_administracion
```

17.1 Uso

Campo	Uso
<code>observaciones_sacad</code>	Correcciones, solicitudes de documentos o aclaraciones de SACAD
<code>observaciones_administracion</code>	Indicaciones administrativas posteriores a aceptación, pagos o comprobaciones

17.2 Revisión

El permiso `solicitudes.review` podrá editar solo campos definidos en una lista blanca, inicialmente:

```
observaciones_sacad  
observaciones_administracion
```

Los cambios de estado quedan reservados a `solicitudes.manage`.

18. Archivo lógico e histórico

El módulo no usará soft deletes.

El histórico se manejará mediante archivo lógico:

```
archived_at
archived_by
archive_reason
```

18.1 Reglas

- El listado normal excluye solicitudes archivadas.
- Solo `solicitudes.manage` puede archivar.
- Archivar no cambia el estado.
- Una solicitud puede estar `CLO` y archivada.
- Una solicitud puede estar `PAG` y archivada.

19. Borrado real

No se usarán soft deletes.

19.1 Propietario

El propietario solo puede borrar sus propias solicitudes si:

- Están en `BORRADOR`.
- No han sido enviadas.
- No tienen trámite institucional iniciado.

19.2 Administración

Un usuario con `solicitudes.manage` puede borrar más casos cuando corresponda institucionalmente.

19.3 Efecto del borrado

Al borrar una solicitud deben borrarse:

- Recursos.
- Visitante.
- Requerimientos.
- Documentos.
- Archivos físicos asociados.

El borrado debe estar protegido por confirmación fuerte.

20. Listados y rendimiento

El listado principal debe ser rápido.

20.1 Listado normal

Debe cargar solo:

- Solicitud.
- Tipo.
- Estado.
- Motivo.
- Propietario.
- Fechas principales.

No debe cargar por defecto:

- Documentos.
- Recursos completos.
- Visitante completo.
- Requerimientos.

20.2 Índices importantes

```
owner_id
created_by
updated_by
tipo_solicitud_id
motivo_id
estatus_id
pais_id
tutor_id
folio_year
folio_year + folio_number
submitted_at
archived_at
fecha_inicio
fecha_fin
```

20.3 Filtros recomendados

- Búsqueda.
 - Año.
 - Estado.
 - Tipo.
 - Motivo.
 - Activas / archivadas / todas.
-

21. Componentes Livewire

21.1 Componentes principales

```
SolicitudesIndex  
SolicitudesEdit  
SolicitudesShow  
EstudiantesAsociadosIndex  
EstudiantesAsociadosEdit
```

21.2 SolicitudesEdit

Se usará un solo componente Livewire para los cuatro pasos.

No se dividirá en componentes por paso.

Internamente deberá organizarse con métodos separados:

```
guardarPaso1()  
guardarVisitante()  
guardarRecursos()  
subirDocumentos()  
validarEnvio()  
enviarSolicitud()
```

22. Servicio principal

Se usará un solo servicio:

```
SolicitudService
```

Responsabilidades:

- Crear borrador.
- Guardar datos generales.
- Guardar visitante.
- Guardar recursos.
- Aceptar política.
- Validar envío.
- Generar folio al enviar.
- Enviar solicitud.
- Cambiar estado.
- Archivar.
- Borrar.

El servicio puede orquestar correos, pero las plantillas deben vivir en Mailables o Notifications dedicadas.

23. Correos y notificaciones

Al enviar una solicitud, el sistema debe enviar correo a:

- Solicitante.
- Grupo de revisión o instancias configuradas.
- Visitante, si aplica y existe correo.
- Otros destinatarios configurables según tipo o política institucional.

La generación del correo debe ocurrir después de confirmar la transacción de envío.

Idealmente se enviará mediante cola.

24. Migración histórica desde SIIAA_10

La migración histórica no forma parte de la primera etapa de construcción.

Se realizará después de:

1. Crear el módulo nuevo.
2. Probarlo.
3. Estabilizarlo.
4. Validar reglas de identidad y catálogos.
5. Diseñar y ejecutar el importador.

24.1 Comando futuro

```
php artisan siiaa10:import-solicitudes
```

24.2 Regla de identidad

Para solicitudes históricas sin `IdentityLink`, el comportamiento base será estricto:

- No migrar la solicitud.
- Generar CSV de omitidos/inconsistencias.

Opcionalmente, en modo explícito, se podrá crear `IdentityLink` faltante usando reglas derivadas:

```
Personas SIIAA: 10000000 + id  
Estudiantes SIIAP: 20000000 + id
```

Siempre generando reporte de alertas.

24.3 Documentos históricos

Los documentos históricos deberán copiarse físicamente a la nueva ruta:

```
documentos/solicitudes/{year}/{solicitud_id}/
```

25. Convenciones técnicas

25.1 Catálogos

Usar siempre:

```
catalogos_items
```

No usar:

```
catalogo_items
```

25.2 Variables de catálogos

Usar prefijo:

```
c_
```

Ejemplos:

```
$c_tipos_solicitud  
$c_motivos  
$c_estatus  
$c_paises  
$c_divisas
```

25.3 Validaciones

Usar preferentemente:

```
Rule::exists
```

contra `catalogos_items`.

25.4 Componentes UI

Usar los componentes Blade UI propios del SIIAA.

26. Orden recomendado de desarrollo

1. Migraciones nuevas.
 2. Modelos y relaciones.
 3. Catálogos/helpers de claves funcionales.
 4. Políticas/permisos.
 5. `SolicitudService`.
 6. `SolicitudesIndex`.
 7. `SolicitudesEdit` Paso 1.
 8. Visitantes y estudiantes asociados.
 9. Recursos.
 10. Documentos.
 11. Paso 4: envío, folio y correos.
 12. `SolicitudesShow`.
 13. Observaciones, revisión administrativa y estados.
 14. Pruebas operativas.
 15. Importador histórico desde SIIAA_10.
-

27. Resultado esperado

El módulo de Solicitudes en SIIAA Laravel 13 deberá ser:

- Más limpio que el módulo anterior.
- Más alineado con identidad institucional.
- Más fácil de mantener.
- Más rápido en listados.
- Más flexible para usuarios reales.
- Más seguro en propiedad y auditoría.
- Preparado para migración histórica.
- Preparado para integración futura con revisión, pagos, proyectos y observaciones avanzadas.

El diseño final prioriza el uso diario y la operación institucional real, evitando complejidad innecesaria sin perder trazabilidad ni consistencia.